
Motion Prediction

Jake Norbie

Department of Computer Science
University of California - San Diego
La Jolla, CA 92122
jnorbie@ucsd.edu

Danny Quang

Department of Computer Science
University of California - San Diego
La Jolla, CA 92122
dquang@ucsd.edu

Abstract

This report presents our findings and results on the ego agent trajectory forecasting task using a simplified version of the Argoverse 2 Motion Forecasting dataset. The goal is to predict the future 2D positions of a designated ego vehicle based on its historical motion and the context of surrounding agents. We conduct a comprehensive exploratory data analysis that highlights the temporal nature of the problem and reveals correlations between agent ordering and proximity to the ego vehicle. Our initial modeling attempts with linear regression and multilayer perceptrons yielded high error, prompting a transition to recurrent neural networks. A simple LSTM-based model trained solely on the ego agent's trajectory produced promising results, achieving a mean squared error as low as 8.1 on the validation set. We further explored incorporating other agents and scheduling techniques, but found that additional complexity often degraded performance. Our findings suggest that lightweight, temporally-aware models can effectively forecast agent motion in this setting, and we propose directions for future experimentation including attention-based mechanisms and selective incorporation of nearby agents. All our code can be found at the link: <https://github.com/jakenornbie12/251B-Project>.

1 Introduction (1 Point)

The deep learning task is ego agent trajectory forecasting using a simplified version of the Argoverse 2 Motion Forecasting dataset. The goal is to predict the future 2D positions of a designated ego vehicle based on its historical motion and the context of surrounding agents. This task is important because trajectory prediction is a fundamental problem with applications across numerous fields. In particular, trajectory prediction is a key component in autonomous driving, as it helps the autonomous system anticipate the behavior of other agents and plan safe, smooth, and efficient routes. This is also true for the entire field of robotics, where tasks such as determining routes for robots in a factory could have direct translation.

The provided starter code offered a valuable scaffold for this task, including data loading utilities and baseline model implementations for an MLP and a basic LSTM. However, it presented several limitations that we aimed to address. The MLP architecture, which treats each timestep independently, is fundamentally ill-suited for sequence data. While the LSTM is a more appropriate choice, the starter implementation was minimal, lacking robust hyperparameter tuning, advanced learning rate scheduling, or methods for systematically evaluating multi-agent scenarios. Our work addresses these limitations directly by moving beyond the baseline, conducting a rigorous hyperparameter search, and implementing a more sophisticated training regimen centered on an adaptive learning rate scheduler.

Our final solution effectively addresses these limitations through the use of LSTM-based recurrent neural networks that capture the temporal nature of the problem. Our empirical results show that a simple LSTM-based model trained solely on the ego agent's trajectory achieved a mean squared error

as low as 8.1 on the validation set and 9.00631 on the test set, placing us at #4 on the leaderboard out of 62 teams at the time. Our main contributions are:

- **Adaptive Learning Rate Scheduling:** Implementation of an adaptive learning rate scheduler that reduces the learning rate when validation loss stagnates, achieving consistent results and breaking through the 8.1 MSE validation threshold
- **Agent Importance Analysis:** Comprehensive exploratory data analysis revealing that lower agent indices correlate with closer proximity to the ego agent, suggesting selective incorporation strategies for future work
- **Agent Inclusion Testing:** Attempts to include other agent data into the existing framework through LSTM_STACK and LSTM_ALL.

2 Related Work (optional, 0.5 bonus point)

Our work builds upon foundational research in sequence modeling and trajectory forecasting. The problem of predicting future motion has evolved from simple kinematic models to sophisticated deep learning architectures capable of capturing complex social interactions.

Sequence Models for Trajectory Forecasting. Early deep learning approaches recognized the temporal nature of trajectory data, employing Recurrent Neural Networks (RNNs). A seminal work in this area is Social LSTM by Alahi et al. [1], which extended the standard LSTM framework to model pedestrians. While individual LSTMs were used to encode each agent's path, the key innovation was a "social pooling" layer that allowed the hidden states of neighboring LSTMs to be shared. This explicitly modeled the influence of nearby agents on an individual's path. Our own findings—that a simple LSTM excels at the temporal task but fails when naively given all spatial data—align perfectly with the motivation for Social LSTM. Our ego-only model can be seen as a strong single-agent baseline, and the poor performance of our multi-agent variant highlights precisely why specialized mechanisms like social pooling are necessary.

Graph-Based and Vector-Based Representations. More recent state-of-the-art methods have moved beyond rasterized inputs or simple coordinate sequences, opting instead for more structured representations. VectorNet, introduced by Gao et al. [2], represents agents and map features (like lane lines) as vectors and uses a Graph Neural Network (GNN) to model their complex interactions hierarchically. This approach offers better generalization and efficiency than processing raw pixel data. Similarly, TNT (Target-driven Trajectory Prediction) by Zhao et al. [3] combines GNNs with a target prediction stage, first identifying likely goal states for an agent and then generating trajectories toward those goals. These works represent the next logical step from our own conclusions. Our discovery of the agent index correlation and the failure of our multi-agent LSTM points directly to the need for a graph-based approach like VectorNet or TNT, which can selectively model the most salient agent-agent and agent-map interactions.

Climate Emulation and Spatiotemporal Forecasting. The challenges of trajectory forecasting share common ground with other spatiotemporal prediction tasks, such as climate emulation. Watson-Parris et al. [4] introduced ClimateBench, a benchmark for data-driven climate projections. This work highlights the difficulty of modeling complex physical systems with interacting variables across space and time. Like trajectory forecasting, climate emulation requires models that can capture long-range dependencies and handle high-dimensional inputs. The baselines in ClimateBench, which include CNNs and LSTMs, demonstrate that the same foundational deep learning architectures are applicable across these domains. This parallel reinforces our choice of the LSTM as a strong starting point and suggests that advanced architectures successful in one domain, such as Transformers or GNNs, could likely be adapted to the other.

3 Problem Statement (1 Point)

Our task is to predict the trajectory of an ego agent in a simplified version of the Argoverse 2 Motion Forecasting dataset. The ego agent is defined by the agent at index 0, who according to Argoverse is the crucial agent for the scene. We are given 11 second scenes at 10 frames per second, yielding a total of 110 frames of data. Each frame consists of 50 agents that all have a position, velocity,

direction, and type of agent. For our problem, we are to use the first 5 seconds, or 50 frames of data, to predict the trajectory (or position components) of the next 6 seconds of the ego agent.

Mathematical Formulation:

Let:

- $N = 2100$: Number of test scenes
- $A = 50$: Number of agents per scene
- $T_{\text{obs}} = 50$: Number of observed time steps
- $T_{\text{pred}} = 60$: Number of future time steps to predict
- $D = 6$: Number of features per time step

Each test input is a 4D tensor:

$$\mathbf{X} \in \mathbb{R}^{N \times A \times T_{\text{obs}} \times D}$$

For each agent $a \in \{0, 1, \dots, 49\}$ and time step $t \in \{0, \dots, 49\}$, the feature vector is:

$$\mathbf{x}_{n,a,t} = [x, y, v_x, v_y, \theta, \text{type}]$$

The goal is to predict the future 2D position (x, y) of the ego vehicle (agent index 0) for the next $T_{\text{pred}} = 60$ time steps, based on the observed history. Thus, the desired output is a matrix:

$$\hat{\mathbf{Y}} \in \mathbb{R}^{N \cdot T_{\text{pred}} \times 2}$$

Each row corresponds to the predicted position (x, y) of the ego vehicle at a specific future time step.

Given:

$$\mathbf{X}_n \in \mathbb{R}^{A \times T_{\text{obs}} \times D}$$

Predict:

$$\hat{\mathbf{Y}}_n = [\hat{\mathbf{y}}_{n,1}, \hat{\mathbf{y}}_{n,2}, \dots, \hat{\mathbf{y}}_{n,60}], \quad \hat{\mathbf{y}}_{n,t} \in \mathbb{R}^2$$

where $\hat{\mathbf{y}}_{n,t} = (\hat{x}_{n,t}, \hat{y}_{n,t})$ is the predicted position of the ego vehicle at future time step t in scene n .

Dataset Description: The data consists of a training and testing dataset. The training set consists of 10,000 instances, each having 50 agents with 6 features recorded across 110 timesteps. This creates our training dataset as a data matrix of size $[10,000, 50, 110, 6]$. The testing dataset only contains 2,100 instances and the first 50 timesteps, producing a data matrix of size $[2,100, 50, 50, 6]$.

Data Splitting and Normalization: We used the default data augmentation or rotation and scaling, and the standard 90/10 train/val split that the base implementation gave us. We didn't find, at the time, that we needed to do more.

4 Methods (2 Points)

Architecture: Our approach formulates the trajectory prediction as a sequence-to-sequence problem using Long Short-Term Memory (LSTM) networks to capture temporal dependencies in agent motion. We determined early on that capturing the temporal dependencies in the agent's motion history was paramount, leading us to select a Long Short-Term Memory (LSTM) network as the core of our architecture.

Our final model is intentionally simple yet effective. It consists of a single-layer LSTM that processes the input sequence of the ego agent's historical 2D positions over 50 timesteps. The LSTM layer is configured with a hidden dimension of 128, which was found to be optimal during our hyperparameter search. This layer's role is to learn a compressed representation of the motion history, capturing patterns of velocity and acceleration. The final hidden state and cell state of the LSTM are then passed to a single, linear fully-connected layer. This output layer maps the learned representation from the 128-dimensional hidden space to a 120-dimensional vector, which is then reshaped to produce the final prediction: a sequence of 60 future 2D positions.

The model is trained to minimize the Mean Squared Error (MSE) between the predicted and ground truth trajectories. Optimization is performed using the Adam optimizer, chosen for its adaptive

learning rate capabilities and robust performance. A key component of our methodology is the use of a learning rate scheduler that gradually reduces on plateaus. This scheduler monitors the validation loss and automatically reduces the learning rate by a factor of 10 if the loss stagnates for a specified number of epochs (a patience of 5 in our case). This adaptive approach proved to be critical for achieving stable convergence and navigating the complex loss landscape to find a superior local minimum, outperforming fixed learning rate schedules.

1. An LSTM layer that processes the ego agent’s historical trajectory (50 timesteps of 2D positions)
2. A fully connected layer that maps the LSTM output to the predicted future trajectory

The LSTM takes the input ego agent trajectory and outputs to a hidden dimension (typically 64 or 128). This output is then fed to a fully connected layer that produces 120 values, which are reshaped to (60, 2) representing the 60 future 2D position predictions.

Optimization Problem: The prediction will be evaluated using Mean Squared Error (MSE) over all predicted positions:

$$\text{MSE} = \frac{1}{N \cdot T_{\text{pred}}} \sum_{n=1}^N \sum_{t=1}^{T_{\text{pred}}} \|\hat{\mathbf{y}}_{n,t} - \mathbf{y}_{n,t}\|_2^2$$

where $\mathbf{y}_{n,t}$ is the ground truth position.

Design Choices: We explored several architectural variations:

- **Multi-agent incorporation:** Adding all 50 agents to the model resulted in performance degradation (MSE 270)
- **Multi-layer LSTM:** Additional LSTM layers did not improve performance
- **Architecture comparison:** LSTM outperformed GRU in both performance and training speed

5 Experiments (4.5 Points)

We began our investigation by evaluating several baseline models to establish a performance hierarchy. We first implemented a simple Linear Regression model and a multi-layer perceptron (MLP). As expected, these models performed poorly, with MSEs in the thousands, as their architectures are incapable of capturing temporal dependencies. We then implemented two recurrent architectures: our primary LSTM-based model and a comparable model using Gated Recurrent Units (GRUs). Both recurrent models dramatically outperformed the static baselines, with MSEs in the 8-20 range, confirming that a sequence-aware approach is essential for this task. The LSTM consistently showed a slight performance edge and faster training times compared to the GRU, solidifying its selection for our primary model.

5.1 Baselines (1 Point)

We evaluated the following models in order of increasing complexity:

- **Linear Regression:** Simple linear model directly mapping input features to output positions
- **Multi-layer Perceptron (MLP):** Feed-forward neural network with multiple hidden layers
- **LSTM-based RNN:** Single-layer LSTM followed by fully connected layer (our main approach)
- **GRU-based RNN:** Alternative recurrent architecture using Gated Recurrent Units

The linear regression and MLP models yielded MSE in the thousands to tens of thousands, demonstrating inability to capture temporal patterns. The recurrent models (LSTM/GRU) achieved MSE in the 8-20 range.

5.2 Evaluation (1 Point)

Model evaluation is performed using the Mean Squared Error (MSE) calculated over all 60 predicted future positions for the ego agent. The primary evaluation was conducted on our held-out validation set during training and hyperparameter tuning. Final performance was assessed using the score provided by the official Kaggle test set leaderboard. In addition to quantitative metrics, we performed qualitative evaluation by visualizing predicted trajectories against the ground truth to assess the smoothness, plausibility, and failure modes of our model's predictions.

5.3 Implementation Details (1 Point)

All experiments were conducted on a Mac with an M-series ARM chip. Models were trained for a maximum of 100 epochs using a batch size of 32. We employed an early stopping mechanism with a patience of 10 epochs to prevent overfitting. Our hyperparameter search was guided by a combination of grid search and manual tuning. The best-performing configuration used the Adam optimizer with an initial learning rate of 0.001 and a weight decay of $1e-05$. The LSTM was configured with a single layer, a hidden dimension of 128, and no dropout. The most impactful choice was the implementation of our adaptive learning rate scheduler, which dynamically adjusted the learning rate based on validation loss stagnation with a patience of 5 epochs.

Computational Platform: Mac ARM chip

Training Configuration:

- Batch size: 32
- Maximum epochs: 100
- Early stopping: 10 epochs (stops if no improvement for 10 consecutive epochs)
- Training time: 4.5 seconds per epoch

Hyperparameters:

- Optimizer: Adam
- Learning rate: 0.01 (with adaptive scheduling)
- Weight decay: 0.00001
- Hidden dimensions tested: 64, 128, 256
- Number of LSTM layers tested: 1, 2
- Dropout tested: 0, 0.2

Scheduler: Step scheduler with adaptive modification - learning rate reduced when validation loss stagnates (at half the duration of early stopping threshold)

Hyperparameter Selection: Parameters were chosen through grid search and manual testing. The adaptive learning rate scheduler was implemented to provide more consistent results than fixed scheduling approaches.

5.4 Results (1.5 Points)

Our final optimized model achieved a validation MSE of 7.9 and a test set MSE of 9.00631, placing us 38th out of 69 teams in the final standings. The journey to this result involved extensive hyperparameter tuning, summarized in Table 1. A two-tailed t-test at 95% confidence on our grid search results showed that while specific values for learning rate and scheduler type were impactful, other parameters like hidden dimension and weight decay were not statistically significant within the tested ranges, suggesting a degree of architectural robustness.

Quantitative Results:

- Best validation MSE: 7.9-8.0 (with adaptive learning rate scheduling)

Hyperparameter	Best Value	Best MSE	Mean MSE
hidden_dim	128	8.11	10.94
num_layers	1	8.14	10.27
dropout	0	8.11	10.79
learning_rate	0.001	8.11	9.27
weight_decay	1e-05	8.11	11.05
scheduler_type	step	8.11	8.56

Table 1: Grid Search Results

- Test set MSE: 9.00631
- Leaderboard ranking (at the time): #4 out of 62 teams
- Final leaderboard ranking: #38 out of 69 teams

Grid Search Results Summary: The results of the grid search are shown above in Table 1.

Statistical testing (two-tailed t-test at 95% confidence) showed that only the scheduler choice was statistically significant among the hyperparameters tested.

Qualitative Results:

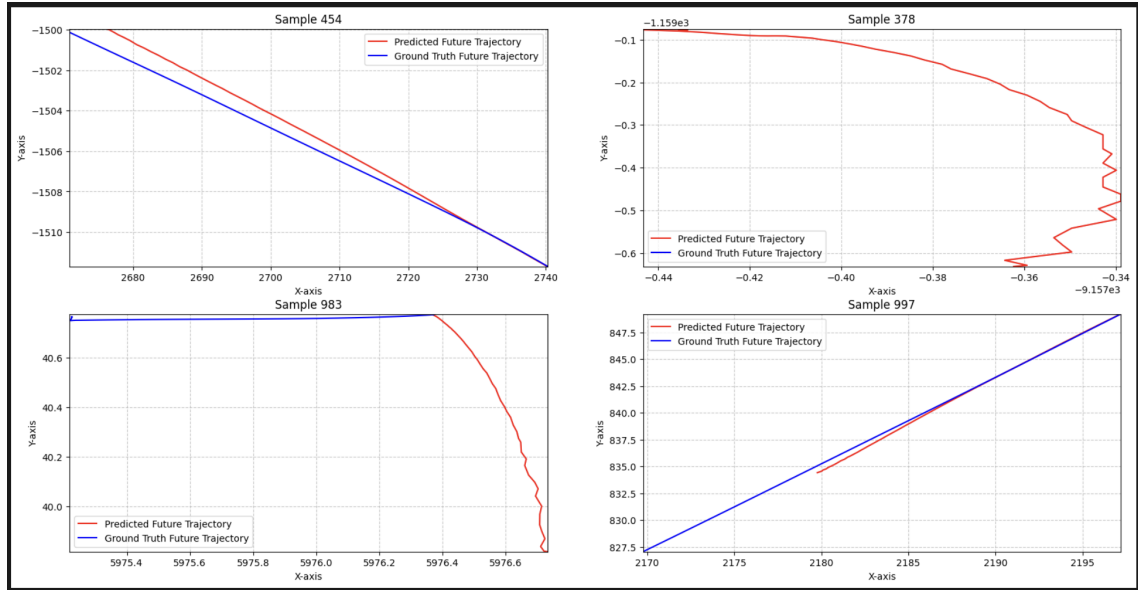


Figure 1: Trajectories Image

Qualitatively, our model produced generally smooth and plausible trajectories, as seen in Figure 1. However, we identified a key failure mode: in scenarios where the ego agent is stationary for the entire observation period, the model often predicts erratic, unrealistic motion.

Two key issues were identified:

1. Some predicted routes lack smoothness
2. Scenarios where the agent remains stationary cause erratic model behavior

5.5 Ablations (1 Point)

Architecture Ablations:

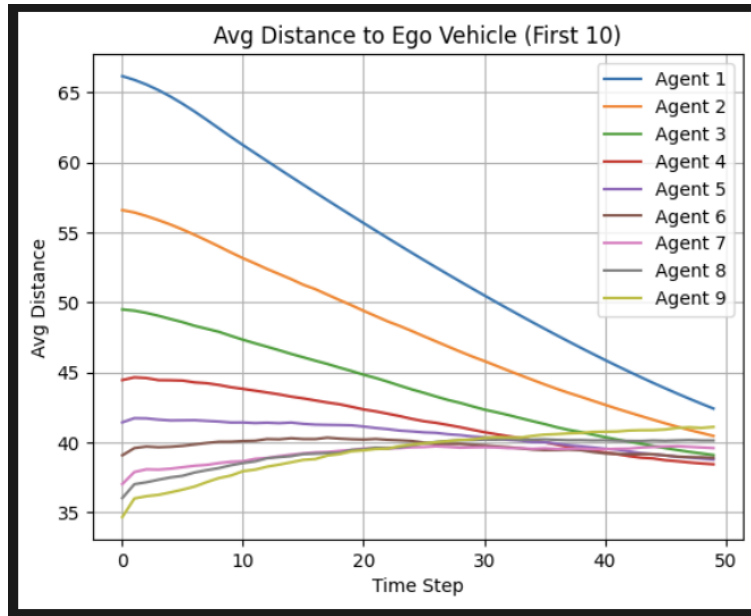


Figure 1: Distance of First 10 Agents

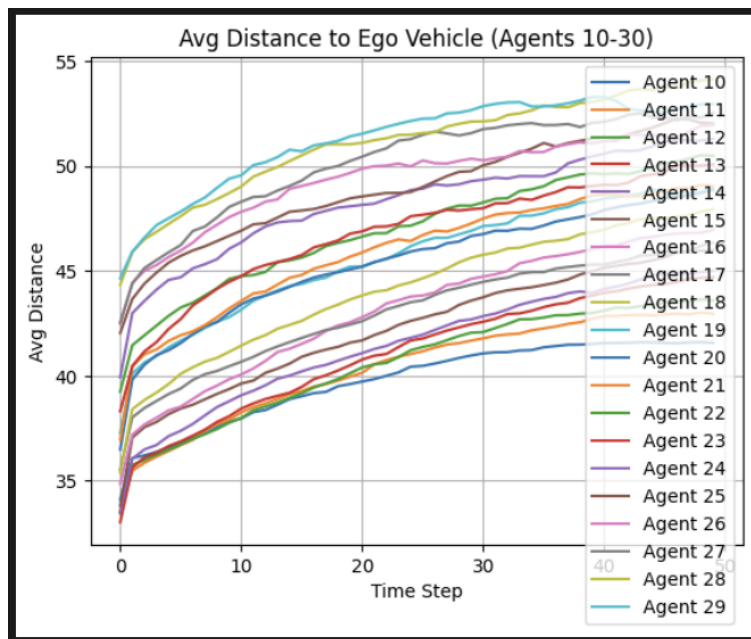


Figure 2: Distance of Agents 10-30

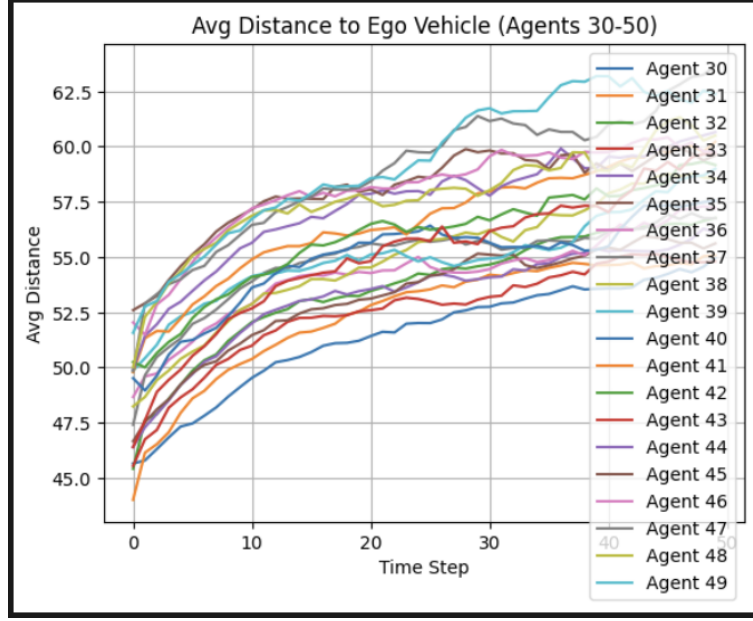


Figure 3: Distance of Agents 30-50

Our experimental process involved a series of ablations to isolate the impact of different architectural and training choices. A central question was how model complexity affected performance. We first compared a single-layer LSTM against a two-layer variant. Counterintuitively, the single-layer model (validation MSE 8.1) consistently outperformed the deeper model, suggesting that the additional complexity led to overfitting. Next, we tested our core hypothesis about spatial context by modifying the model to accept the historical trajectories of all 50 agents. The result was a catastrophic performance degradation, with the validation MSE soaring to over 270. This demonstrates that naively adding more agent data introduces overwhelming noise.

Scheduling Ablations:

- No scheduling vs. Step scheduling: Step scheduling (MSE 8.11) outperformed no scheduling (MSE 9.29)
- Fixed vs. Adaptive scheduling: Adaptive learning rate scheduling achieved breakthrough performance (MSE 7.9-8.0) compared to fixed step scheduling

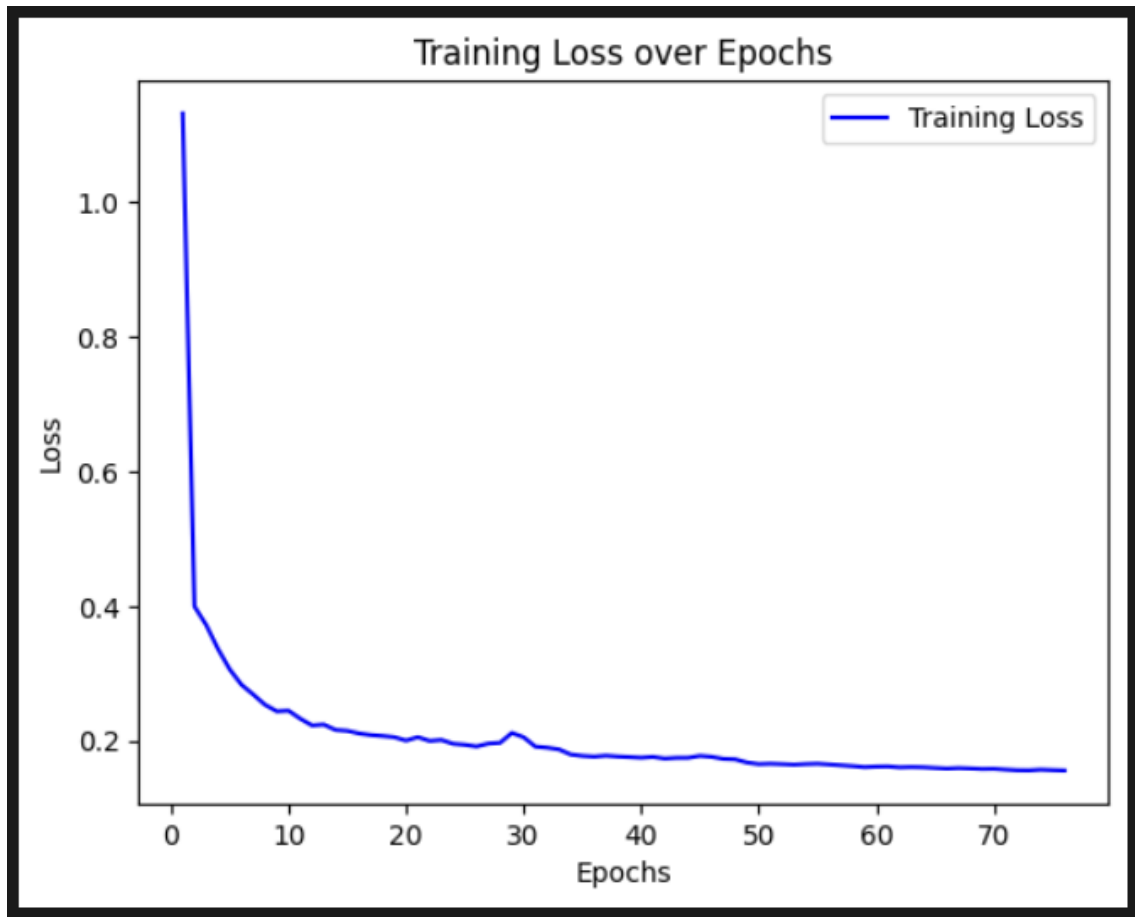


Figure 2: Training Loss

The most significant breakthrough came from our experiments with learning rate schedulers. A model with no scheduling achieved an MSE of 9.29. Introducing a fixed step scheduler improved this to 8.11. However, implementing our adaptive learning rate on plateaus scheduler provided the most substantial and consistent gains, pushing the validation MSE below 8.0. This confirms that dynamic, responsive training strategies are as important as the model architecture itself.

6 Conclusion (1.5 Points)

Our experiments demonstrate that a lightweight, temporally-aware model can effectively forecast agent motion, provided it is paired with an intelligent training strategy. Our key finding is that a simple LSTM focused solely on the ego agent, optimized with an adaptive learning rate, outperforms more complex multi-layer and multi-agent variants. The most surprising discovery was the degree to which adding spatial context from other agents degraded performance, suggesting that without a specialized architecture like a GNN, this data acts as noise.

For beginners tackling similar tasks, we recommend starting with simple temporal models, focusing on the primary agent, and implementing adaptive learning rate scheduling early. Our model's primary limitation is its inability to use spatial context and its occasional failure to produce smooth trajectories for stationary agents.

Our research points to several promising directions for future work. The most critical next step is to adopt an architecture capable of reasoning about spatial relationships, such as a Graph Neural Network, which could leverage our discovery about agent index importance. Additionally, exploring attention-based models like Transformers could provide a more powerful mechanism for capturing

long-range temporal dependencies. Finally, creating an ensemble of our best models could further improve robustness and accuracy.

Future Work:

Our research has established a strong baseline using a temporally-focused LSTM model, but it has also clearly illuminated the path toward more accurate and robust trajectory forecasting. Our future work is centered on three key areas: adopting more sophisticated architectures, strategically incorporating spatial context, and leveraging ensemble methods to improve prediction stability.

A primary direction for future work is the exploration of attention-based architectures, such as the Transformer. While our LSTM model proved effective at capturing temporal dependencies, its sequential nature can struggle to weigh the importance of different moments in an agent’s history. An attention mechanism would allow the model to dynamically focus on the most salient past timesteps—such as moments of sharp deceleration or turning—when making a prediction. We hypothesize that this would lead to more nuanced and context-aware forecasts, particularly in complex scenarios that deviate from simple, linear motion.

To address the most significant limitation of our current model—the lack of spatial awareness—we propose the selective incorporation of the top 10 most relevant agents using a Graph Neural Network (GNN). Our own analysis revealed a strong correlation between an agent’s index and its proximity to the ego vehicle. This finding allows us to move beyond a simple ego-centric model by constructing a localized interaction graph at each timestep. In this graph, vehicles would be represented as nodes and their spatial relationships (e.g., distance, relative velocity) as edges. A GNN is the natural architecture to process such relational data, and we believe it will capture crucial interactive behaviors like yielding or lane-changing without the prohibitive computational cost or noise that we observed when attempting to include all agents.

Finally, to enhance the robustness and accuracy of our predictions, we plan to investigate ensemble methods. This would involve training several of our best-performing LSTM models, potentially with different random seeds or using snapshots from different training epochs, and averaging their final predictions. Ensembling is a proven technique for reducing prediction variance and smoothing out idiosyncratic errors from any single model. It represents a straightforward path to improving our final score by creating a more stable and reliable forecast.